

COMP-3150 CHAPTERs 8: LECTURE NOTES (Ch. 8)

- Chapter 8:
The Relational Algebra and The Relational Calculus

The following questions on **The Relational Algebra and The Relational Calculus** discussed in Chapter 8 of Comp 3150 text book, Chapters 8, Comp 3150 posted course slide notes, are in-class questions for students to ponder and answer as I teach.

- The answers to the questions are found also by reviewing the Comp 3150, posted power point slide notes for Chapters 8 and being in class.
- Students are advised to review Chapter 8 of course book and Comp 3150 posted slide notes before and after each class.
- I will also go over the Slide notes in class with examples and integrate them into the class lectures, which are also posted in the More course materials link on black

board with any links to recorded live lectures.

1. What is Relational Algebra?

- It is one of the two formal languages for the relational model (the relational algebra and relational calculus) that SQL is based on.
- Relational Algebra is the procedural formal language while relation calculus is the declarative formal language.
- These formal languages provide operators for implementing the same types of queries we executed with SQL in Chapters 6 and 7.

i) What are Relational Algebra Operators? (slides 1 to 38 of Ch. 8 notes)

- They are:

(a) Unary Relational Operators

- i) Select : σ (sigma)
- ii) Project : π (pi)
- iii) Rename : ρ (rho)

(b) Binary Relational Operations

- iv) Join : $\bowtie$ (bow-tie)

v) Division : $\div$

(c) Relational Algebra Operations From Set Theory

vi) Union : $\cup$

vii) Intersection : $\cap$

viii) Difference (or minus) : $-$

ix) Cartesian Product : $\times$

d) Additional Relational Operations

x) Aggregate Functions and Grouping: $\bowtie$ (Script)

2. What is Relational Calculus?

- The declarative formal query language that SQL is based on.
- A version of Relational Calculus we are using is Tuple Relation Calculus (TRC).

Discussions

1. What is Relational Algebra?

Check the Ch. 8 slides pages 1 to 42 for a discussion of the relational algebra and its operators.

- An Example Introduction to both of these languages in comparison to SQL is given in answer to the following question:

Example Query 1: (Simple SPJ)

Get the employee Lname and Fname whose department number is 4. Write this query in:

- (i) SQL
- (ii) Relational Algebra
- (iii) Relational Calculus
- (iv) Show the result retrieved for this query

Solutions

- (i) SQL

```
SELECT LNAME, FNAME  
FROM EMPLOYEE  
WHERE DNO=4;
```

- (ii) Relational Algebra

```
 $\pi_{Lname, Fname} (\sigma_{Dno=4} (Employee))$ 
```

(iii) Relational Calculus

$\{t \mid \text{Cond}(t)\}$

where t is set of attributes of relations in the database and $\text{Cond}(t)$ is a formula that involves attributes of t that must evaluate to TRUE for the tuple to be retrieved as result.

The query above in Relational Calculus is:

$\{e.\text{Lname}, e.\text{Fname} \mid \text{Employee}(e) \text{ and } e.\text{Dno} = 4\}$

(iv) Show the result of the query:

```
SELECT LNAME, FNAME
FROM EMPLOYEE
WHERE DNO=4;
```

LNAME	FNAME
Zelaya	Alicia
Wallace	Jennifer
Jabbar	Ahmad

Example Query 1 using rho : Rename using ρ (rho) the result of example query 1 above such that the Lname is renamed Last_name and Fname is renamed First_name. Query 1 is: Get the employee Lname and Fname whose department number is 4.

- $\rho_{\text{(Last_name, First_name)}} (\pi_{\text{Lname, Fname}} (\sigma_{\text{Dno=4}} (\text{Employee})))$

■ Example Query on Slide 11 using rho: To list each employee's first and last name and salary as shown on slide 11 and below with the left arrow (assignment), but using ρ the following relational expression with ρ is used:

- $R(\text{Last_name, First_name, Salary}) \leftarrow$

$$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$$

- $\rho_{R(\text{Last_name, First_name, Salary})} (\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE}))$

Other Relational Algebra Query Examples

An example of a query that requires aggregate function for summarizing stored attributes of a database relation (such as query) of relation Employee is: $\sigma, \mathfrak{A}, \rho, \div, \bowtie$

Example Query 1a: As on Slide 16 showing use of set operation union U. Get SSN of all employees who either work in department 5 or directly supervises an employee who works in department 5.

- A broken down solution for this query is provided on slide 17.
- However, an alternative solution for any relational algebra query is give a one expression solution for the query as below:

$$\pi_{SSN}(\sigma_{Dno=5} (Employee)) \cup \pi_{Super_Ssn} (\sigma_{Dno=5} (Employee))$$

Alternative Solution of Query 1a given on Slide 17:

Example: To retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or directly supervise an employee who works in department 5 (RESULT2 below)

- We can use the UNION operation as follows:

$$DEP5_EMPS \leftarrow \sigma_{DNO=5} (EMPLOYEE)$$

$$RESULT1 \leftarrow \pi_{SSN}(DEP5_EMPS)$$

$$RESULT2(SSN) \leftarrow \pi_{SUPERSSN}(DEP5_EMPS)$$

$$RESULT \leftarrow RESULT1 \cup RESULT2$$

Example Query 1b: Similar to query on Slide 25-29 solved using cross product, but now solved with join . Get the female employees (Lname, Fname, Ssn) who have dependents. Solve as a relational algebra expression using join.

$$\pi_{Lname, Fname, Ssn} (\sigma_{Sex = 'F'} (Employee \bowtie_{Ssn=Essn} Dependent))$$

Example Query 2: (including aggregate function)

Get the maximum Salary of all female employees

- This query of Example 2 can be answered in SQL as:

SELECT max(Salary)

FROM Employee

WHERE Sex = 'F';

- The Relational algebra query solution for Example 2 is:

$\mathfrak{J}_{\text{maximum salary}}(\sigma_{\text{Sex} = \text{'F'}}(\text{Employee}))$

- This is because as presented on Slide 40 of Ch. 8 notes, the general format for a relational algebra query is:

$\langle \text{grouping attributes} \rangle \mathfrak{J} \langle \text{function list} \rangle (R)$

where $\langle \text{grouping attributes} \rangle$ is a list of attributes of relation R and $\langle \text{function list} \rangle$ is a list of ($\langle \text{function} \rangle \langle \text{attribute} \rangle$) pair.

Function is one of SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT.

- The result has the grouping attributes plus one attribute for each element in the function list.

Example Query 3: (include aggregate functions with group by attribute(s))

Retrieve the maximum Salary of female employees in each department.

(i) SQL: SELECT Dno, max(Salary)
FROM Employee
WHERE Sex = 'F'
GROUP by Dno;

(ii) Relational Algebra:
Dno $\Join$ maximum salary ($\sigma_{\text{Sex} = 'F'}(\text{Employee})$)

- See slides 41 and 42 of ch. 8 notes for more related examples and the results retrieved by these queries on the COMPANY database instance of Slide 10.
- (iii) The result of Query 3 on the COMPANY database is:

DNO MAX(SALARY)

1 55000
5 40000
4 43000

Example Query 4: (including use of join $\Join$ operator)

List the names of employees who have a dependent with the same first name as themselves. Write both the SQL and relational algebra queries for this English query.

Some symbols for solving queries you may copy and reuse are: π , σ , ρ , $\div$, $\bowtie$

Some more Relational Algebra Query examples: (for in-class work) similar to those in assn 3,

Solution to Example Query 4:

SQL Solution:

```
SELECT e.Fname, e.Lname
FROM Employee e, Dependent d
WHERE e.Ssn = d.Essn
AND d.Dependent_name = e.Fname;
```

RELATIONAL ALGEBRA Solution:

```
E  $\leftarrow$  ( $\sigma_{\text{Fname} = \text{Dependent\_name}}$  (EMPLOYEE  $\bowtie$   $\sigma_{\text{Ssn} = \text{Essn}}$  (DEPENDENT))))
R  $\leftarrow$   $\pi_{\text{Lname}, \text{Fname}}$  ( E )
```

- You can also give the solution above in one Relational algebra expression not broken down but in one single relational algebra expression as:

```
 $\pi_{\text{Lname}, \text{Fname}}$  ( $\sigma_{\text{Fname} = \text{Dependent\_name}}$  (EMPLOYEE  $\bowtie$   $\sigma_{\text{Ssn} = \text{Essn}}$  (DEPENDENT))))
```

Example Query 5: (including use of Division operator $\div$)

Retrieve the names of employees who work on all the projects that 'John Smith' works on. See discussions on slides 36 to 38 of Ch. 8 slides.

- The result of Query 5 will be:

(Essn with Pnos worked by all employees) $\div$ (Pnos worked by John Smith)

$\pi_{\text{Essn, Pno}} (\text{WORKS_ON}) \div (\pi_{\text{Pno}} (\sigma_{\text{Lname}='Smith' \text{ and } \text{Fname}='John'} ($
 $\text{WORKS_ON} \bowtie_{\text{Essn=Ssn}} \text{EMPLOYEE})))$

- This is the end of Relational Algebra and next we continue to discuss the Relational Calculus for querying databases starting from Slide 43.

Refer to slides 43 to 48 of posted Ch 8 Course notes for a discussion of the Relational Calculus, the third database language discussed in the Course for querying databases. Review Page 272 of book too.

2. What is Relational Calculus

- See slides 42 to 48 of Ch. 8 for discussions of Relational calculus and its language. The two other languages discussed are SQL and Relational algebra.

- While Relational algebra expressions are more procedural, Relational calculus expressions are more declarative. These are formal query languages for the relational model.
- A calculus expression specifies what is to be retrieved rather than how to retrieve it as done by the relational algebra.
- The version of the relational calculus being discussed is tuple relational calculus (TRC) which has attribute scope.
- Slide 45 of Ch 8 notes summarize the relational calculus language as:

- A relational Calculus query is expressed as :

{t | Cond(t)}

Where t is a tuple variable and only attributes of a tuple t (such as $t_1.A_j$, $t_2.A_k$) are used in the query and in the formula representing **Cond(t)** that must be **TRUE** for a tuple t to be retrieved as a result.

- Cond(t) is an atom or formula of the form:
 - i) Relation (t_i), eg. Employee(t)
 - ii) $t_1.A \text{ op } t_1.B$, eg. E.ssn = d.Essn
 - iii) $t_1.A \text{ op } C$, eg. Salary > 3000
 - iv) A formula F, is made up of atoms and atoms are connected with logical operators (ops) and quantifiers ($\exists$, $\forall$), eg $F_1 \text{ AND } F_2$, $F_1 \text{ OR } F_2$, NOT(F_1), $\exists(F)$, and $(\forall)(F)$.

Example Query 6:

Write the Relational Calculus query for the query:

List the names and addresses of all employees with salary > 30,000.

i) Relational Calculus Solution:

$\{e.Fname, e.Lname, e.Address \mid \text{Employee}(e) \text{ and } e.Salary > 30000\}$

ii) Relational Algebra Solution:

$\pi_{Fname, Lname, Address}(\sigma_{Salary > 30000}(\text{Employee}))$

iii) SQL Solution:

```
SELECT Fname, Lname, Address
FROM Employee
WHERE Salary > 30000;
```

iv) Result retrieved by this query is:

```
SELECT Fname, Lname, Address
FROM Employee
WHERE Salary > 30000;
```

FNAME	LNAME	ADDRESS
-------	-------	---------

Franklin	Wong	638 Voss, Houston, TX
Jennifer	Wallace	291 Berry, Bellaire, TX
Ramesh	Narayan	973 Fire Oak, Humble, TX
James	Borg	450 Stone, Houston, TX

Note that this query is simple and does not involve join or division (the binary relational operators) needed when you are posing a query that requires more than one database table to answer. We shall conclude in the next class.

RELATIONAL CALCULUS CONTINUES:

- We continue the discussion of relational calculus for querying databases as summarized on slides 43 to 48 of web posted course notes chapter 8.
- Earlier, we posed a relational calculus (RC) query for a simple query:
 - o List the names and address of all employees with Salary > 30000.

As:

{e.fname, e.lname, e.address | employee(e) and e.Salary > 30000}

- The query above is simple because its results can be retrieved from only one table employee and join operation is not involved.
- Let us also discuss example queries with the following more complex features:
 - (1) Queries involving more than one table and those needing join operation (done with existential quantifier. $\exists x$).
 - (2) Queries needing universal quantification ($\forall x$) for all x. This implements the equivalent of division $\div$, we saw in Relational algebra querying.
- These examples and the use of existential and universal quantifiers in RC are discussed around pages 272 to 274 of course book and students are encouraged to review course book for more in-depth understanding of this topic and chapter 17 indexing topic to be discussed next in class.
- For now, I will continue to give RC example queries in categories (1) Join and (2) Division. The example query numbering begins now again with Query 1 for RC and these numbers may correspond to the numbering for similar queries in the course text book.

- Query 1: List the name and address of all employees who work for the Research department.

Note: This is an example involving more than one table and requires join of the two tables. All attributes to the left of the bar in the RC query are free variables to be retrieved as result.

- Inside the RC query, any other attribute needed in the formula on the right hand side of the bar in $\{ t \mid \text{Cond}(t) \}$, which is not already a free variable instantiated to a table in the database needs to be bound to a table in the database using existential quantifier ($\exists x$).

Query 1: RC Solution to Query 1 above is

Solution:

$\{t.Fname, t.Lname, t.Address \mid \text{Employee}(t) \text{ AND } (\exists d)$
 $(\text{Department}(d) \text{ AND } d.Dname = \text{'Research'} \text{ AND } d.Dnumber =$
 $t.Dno)\}$

Query 2: For every project located in 'Stafford' list the project number, the controlling department number and the department manager's last name, birthdate and address.

Solution Q2:

$\{p.Pnumber, p.Dnum, m.Lname, m.Bdate, m.Address \mid PROJECT(p) \text{ AND } EMPLOYEE(m) \text{ AND } p.location = 'stafford' \text{ AND } (\exists d) (DEPARTMENT(d) \text{ AND } p.Dnum = d.number \text{ AND } d.Mgr_ssn = m.ssn))\}$

In query2, note that tuple variables 'p' and 'm' are free and tuple variable d has to be introduced with $(\exists d)$

Query 3': List the name of each employee who works on some projects controlled by dept 5.

Note: This is a variant of Query 3 in which some $(\exists x)$ of query 3' is changed to all $(\forall x)$ in query 3 (shown next).

Solution to Query 3':

$\{e.Lname, e.Fname \mid Employee(e) \text{ AND } ((\exists x) (\exists w) (PROJECT(x) \text{ AND } WORKS_ON(w) \text{ AND } x.Dnum = 5 \text{ AND } w.Essn = e.ssn \text{ AND } x.Pnumber = w.Pno)))\}$

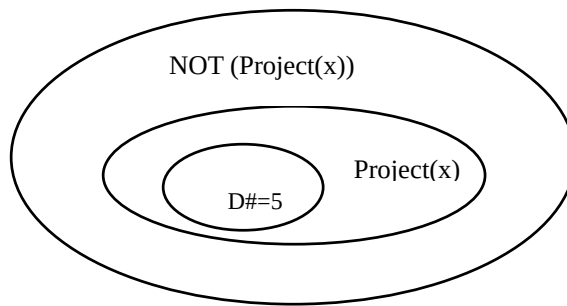
- Now the example using $(\forall x)$ quantifier that is a version of Query 3' is next.

Query 3: List the names of employees who work on all the projects controlled by department number 5.

Solution: $\{e.Lname, e.Fname \mid Employee(e) \text{ AND } ((\forall x) (\text{NOT PROJECT}(x) \text{ OR NOT}(x.Dnum=5) \text{ OR } ((\exists w) (\text{WORKS_ON}(w) \text{ AND } w.Essn = e.ssn \text{ AND } x.Pnumber = w.Pno))))) \}$

- Explanations of the solution involving $(\forall x)$ follows.
- To be TRUE, $\forall x(F)$ requires the inner formula F be TRUE for all tuples in the universe.
- Thus, we exclude from the universal quantification all tuples we are not interested in, by making condition TRUE for such tuples.
- Thus, we want employees who work on all projects controlled by dept #5.
- Tuples in the database universe that are not in the PROJECT relation R of interest are first excluded by making them TRUE in F with NOT PROJECT (x) so that it moves on to PROJECT tuples.

- In PROJECT table, we again exclude tuples of PROJECT not in dept 5 with NOT ($x.Dnum = 5$).



- Now, we are left with all projects in dept 5, we need to join the Works_on table introduced with $(\exists w)$ with Employee (e) to get all employees who worked on all those project x controlled by dept #5 which are project {1, 2, 3}.
- That final condition is specified in Q3 solution last.
- Note that formula of the form (NOT ($x.Dnum = 5$) or F2) is equivalent to If ($x.Dnum = 5$) Then F2.
- Thus, with respect to Q3, a tuple x in project relation is either in $x.Dnum \neq 5$ or it must satisfy F2 (the final condition in the $\forall x(F)$ formula. This concludes Ch 8.
- Next, we discuss Ch 17.